

XIV Maratona UnB de Programação

September 25, 2026

Contest Session

This problem set contains **14** problems; pages are numbered from 1 to 33.

v1.0

Problem A — Árvore Six Seven

O Natal está próximo, precisamos começar nossas preparações desde já.

Um dos grandes símbolos do Natal é a árvore six seven , uma árvore (um grafo não direcionado, conexo e acíclico) para a qual a distância entre quaisquer duas folhas (vértices com menos de dois vizinhos) distintas é 6 ou 7. ♀

A distância entre dois vértices é definida como o número de arestas no caminho entre eles.

Por exemplo, se o caminho entre duas folhas contém 6 arestas, tudo certo: six seven . Se o caminho possui 7 arestas, também está valendo: aura farm completo 😊.

Por outro lado, se a distância entre duas folhas for 5 ou 8, a árvore perde toda a sua aura ☹ está fora da resenha.

Sua missão é, dado um inteiro N , construir uma árvore six seven com exatamente N vértices ou dizer que ela não existe.

Entrada

A entrada consiste de um único inteiro N ($1 \leq N \leq 10^5$).

Saída

Caso não exista uma árvore six seven com N vértices, imprima -1 .

Caso exista, imprima K ($0 \leq K \leq 10^5$), o número de arestas de sua árvore, em uma linha. Em seguida, imprima K linhas contendo dois inteiros u e v ($1 \leq u, v \leq N$) cada, o que representa uma aresta entre os vértices u e v .

Exemplos

Entrada

```
7
```

Saída

```
6
1 2
2 3
3 4
4 5
5 6
6 7
```

Entrada

```
2
```

Saída

-1

Problem B — Bom Balanço de Divisores

Dizemos que um número n possui um bom balanço de divisores se, ao considerar todos os seus divisores d (exceto o próprio n) e dividi-los por n , a soma for exatamente 1. Formalmente, n tem divisores balanceados se e somente se $1 = \sum_{d|n, d \neq n} \frac{d}{n}$.

Pedimos que você conte quantos números até N possuem um bom balanço de divisores.

Entrada

A entrada consiste em um único inteiro N ($1 \leq N < 2^{63}$).

Saída

Imprima um único inteiro: a quantidade de números até N que possuem um bom balanço de divisores.

Exemplos

Entrada

5

Saída

0

Entrada

6

Saída

1

Explicação

Os divisores de 6 são 1, 2, 3 e 6. Podemos ver que $1/6 + 2/6 + 3/6 = 1$, portanto, 6 possui um bom balanço de divisores.

Entrada

30

Saída

2

Entrada

9223372036854775807

Saída

8

Entrada

10000000000000000000

Saída

7

Entrada

10000

Saída

4

Problem C — Campinho

João Vitor gosta de jogar “Campinho”: um “fantasy game” em que você compra jogadores de futebol para o seu time a fim de alcançar a maior pontuação possível. A pontuação do time é simplesmente a soma das pontuações de cada jogador.

No jogo, cada um dos jogadores disponíveis tem uma posição fixa, representada por uma letra: goleiro (G), zagueiro (Z), lateral (L), meio-campista (M) ou atacante (A). Cada jogador também tem um preço (em reais, com duas casas decimais). A pontuação de cada jogador depende de como ele irá se sair na próxima partida, mas, devido ao conhecimento avançado de João em Analytics, ele também sabe a pontuação que cada jogador receberá (também um número com duas casas decimais).

Sendo um apreciador de táticas modernas, João usará a formação 4-3-3, o que significa que seu time consistirá de 1 goleiro (G), 2 zagueiros (Z), 2 laterais (L), 3 meio-campistas (M) e 3 atacantes (A). Além disso, ele tem apenas X reais para gastar com jogadores. Com base nos preços e pontuações informados para cada jogador, ele consegue montar um time com a formação 4-3-3? Se sim, qual é a pontuação máxima que ele pode alcançar?

Entrada

A primeira linha da entrada contém dois valores separados por um espaço: N e X . N é o número de jogadores disponíveis para compra ($1 \leq N \leq 200$). X é o valor, em reais, que João tem para gastar, com exatamente duas casas decimais ($0.00 \leq X \leq 200.00$).

As próximas N linhas descrevem os jogadores disponíveis. A i -ésima dessas linhas contém três valores separados por um espaço: C_i , P_i e S_i . C_i é um caractere maiúsculo representando a posição do i -ésimo jogador: G, Z, L, M ou A. P_i e S_i têm ambos exatamente duas casas decimais ($0.00 \leq P_i, S_i \leq 50.00$) e representam, respectivamente, o preço e a pontuação desse jogador.

Saída

Se João não conseguir montar um time com a formação 4-3-3, imprima -1 . Caso contrário, imprima a pontuação máxima que ele pode alcançar, com exatamente duas casas decimais.

Exemplos

Entrada

```
1 200.00
G 0.00 50.00
```

Saída

```
-1
```

Explicação

No primeiro caso de teste, não é possível montar um time com a formação 4-3-3.

Entrada

```
18 101.00
G 0.00 0.00
G 10.00 10.00
Z 5.00 2.00
Z 5.00 2.00
Z 7.00 1.00
Z 7.00 1.00
L 50.00 18.00
L 50.00 18.00
L 20.00 4.00
L 20.00 4.00
M 1.00 1.00
M 2.00 2.00
M 3.00 3.00
M 10.00 10.00
A 15.00 15.00
A 12.00 18.00
A 3.00 2.00
A 0.00 0.00
```

Saída

```
69.00
```

Explicação

No segundo caso de teste, o melhor time custa exatamente 101.00 e consiste no goleiro custando 10.00; nos dois zagueiros custando 5.00 cada; nos dois laterais custando 20.00 cada; nos meio-campistas custando 1.00, 3.00 e 10.00; e nos atacantes custando 15.00, 12.00 e 0.00. As pontuações deles são, respectivamente, 10.00, 2.00, 2.00, 4.00, 4.00, 1.00, 3.00, 10.00, 15.00, 18.00 e 0.00, totalizando uma pontuação de 69.00.

Problem D — Desvendando as Formigas

Wilson é um grande admirador de formigas e quer estudar melhor uma espécie, buscando aprender mais sobre seus hábitos.

Para isso, ele observa uma trilha reta, dividida em posições identificadas por números inteiros. Cada posição pode ser ocupada por, no máximo, uma formiga. No início do estudo, não há formigas na trilha.

Ao longo do estudo, acontecem três tipos de eventos (numerados de 1 a 3), cada um associado a um número (denotado por x):

- 1 x : uma nova formiga entra na posição x . Se já existia uma formiga nesta posição, a formiga antiga sai da trilha, sendo substituída pela nova.
- 2 x : a posição de toda formiga na trilha é incrementada em x . Ou seja, se $x > 0$, as formigas se deslocam x posições para a direita; mas se $x < 0$, elas se deslocam x posições para a esquerda.
- 3 x : Wilson conta quantas formigas na trilha estão à esquerda da posição x — ou seja, quantas posições p tais que $p < x$ possuem uma formiga.

Replique o estudo realizado por Wilson, processando os eventos na ordem em que ocorreram e imprimindo a quantidade contada por ele em cada evento do tipo 3.

Entrada

A primeira linha contém um inteiro Q ($1 \leq Q \leq 5 \cdot 10^5$): o número de eventos.

Cada uma das próximas Q linhas contém dois inteiros t e v , com $t \in \{1, 2, 3\}$ e $|v| < 2^{63}$. t é o tipo do evento e v é o valor do número x associado a este evento, mas codificado.

Para obter o valor real de x , você deve tomar $x = v \oplus r$, onde:

- r é o valor do último resultado imprimido para um evento do tipo 3. Se nenhum evento do tipo 3 tiver ocorrido até o evento atual, considere que $r = 0$ e, portanto, $x = v$.
- $\oplus$ é operação de XOR bit a bit entre a representação de números inteiros em 64 bits.
- A representação de um número a (tal que $|a| < 2^{63}$) em 64 bits corresponde aos 64 dígitos binários menos significativos do número a , se $a \geq 0$. Se $a < 0$, a representação corresponde aos mesmos dígitos, mas de $2^{64} + a$.

Isso significa que o valor de x pode ser obtido pela representação binária resultante da operação $v \oplus r$. Nessa representação, se o 64-ésimo bit menos significativo possuir valor 0, os dígitos binários representam exatamente o número x . Caso contrário, eles representam $2^{64} + x$. É garantido que $|x| \leq 10^9$.

Além disso, também é garantido que existe pelo menos um evento do tipo 3.

Saída

Para cada evento do tipo 3, imprima uma linha contendo um único inteiro: a quantidade de formigas na trilha em posições à esquerda de x .

Observações

Note que, se sua solução imprimir um valor incorreto para um evento do tipo 3, para os eventos seguintes o valor de x pode ser decodificado incorretamente (considerando que a solução atualiza r de acordo). Perceba que isso pode resultar em vereditos diferentes de Wrong Answer, mesmo que a solução de fato tenha imprimido um resultado incorreto.

Exemplos

Entrada

```
12
1 3
1 -2
3 3
1 3
1 3
2 -3
3 -2
1 -4
2 7
3 6
3 -12
3 100
```

Saída

```
1
2
2
0
3
```

Explicação

Inicialmente, não há formigas na trilha e $r = 0$.

Nos dois primeiros eventos, como ainda não houve nenhum evento do tipo 3, obtemos x diretamente com $x = v$. Assim, entram formigas nas posições 3 e -2 . A trilha contém formigas nas posições $\{-2, 3\}$.

No terceiro evento, temos $x = 3$. Existe apenas uma formiga estritamente à esquerda de 3, na posição -2 . Portanto, imprimimos 1 e passamos a ter $r = 1$.

No evento seguinte, o valor fornecido é $v = 3$. Como $r = 1$, o valor real é $x = 3 \oplus 1 = 2$. Portanto, uma nova formiga entra na posição 2, e as posições ocupadas passam a ser $\{-2, 2, 3\}$.

O evento seguinte é novamente 1 3, que também é decodificado como $x = 2$. Como já existe uma formiga nessa posição, ela é substituída pela nova, e o conjunto de posições ocupadas não muda.

No sexto evento, temos $v = -3$ e $r = 1$. Interpretando o XOR nas representações de 64

bits, obtemos $x = (-3) \oplus 1 = -4$. Todas as formigas se deslocam, então, 4 posições para a esquerda. Suas novas posições são $\{-6, -2, -1\}$.

No próximo evento, $v = -2$, e $x = (-2) \oplus 1 = -1$. Existem duas formigas estritamente à esquerda de -1 , nas posições -6 e -2 . A formiga que está exatamente em -1 não é contada. Portanto, imprimimos 2 e r passa a ser 2.

Em seguida, $1 - 4$ é decodificado como $x = -2$. Já existe uma formiga nessa posição, então ela é apenas substituída.

No evento 2 7, temos $x = 7 \oplus 2 = 5$. Todas as formigas se deslocam 5 posições para a direita, passando para $\{-1, 3, 4\}$.

No evento 3 6, $x = 6 \oplus 2 = 4$. As formigas nas posições -1 e 3 estão à esquerda de 4, enquanto a formiga na própria posição 4 não é contada. Portanto, imprimimos 2.

No evento seguinte, $x = (-12) \oplus 2 = -10$. Não há nenhuma formiga à esquerda de -10 , então imprimimos 0. Com isso, r volta a ser 0.

Por fim, como $r = 0$, o último valor não é alterado: $x = 100$. As três formigas estão à esquerda dessa posição, então imprimimos 3.

Problem E — Encontrando Ocorrências

Quem nunca virou a noite antes de uma prova importante? Especialmente quando há uma disciplina específica que você simplesmente não consegue entender... Essa é a situação de Carlos Brasil: ele está estudando para sua próxima prova, e há um certo tópico, representado por uma palavra-chave K , que o assombra.

Ele precisa ler N livros para a prova. Como lê muito rápido, ele pode representar o i -ésimo desses livros por uma única string B_i . Mas, como está com sono, ele começou a ter algumas alucinações, nas quais, para cada string B_i , no máximo uma ocorrência da palavra-chave K em B_i fica destacada para ele.

Note que uma ocorrência de uma string S em uma string T é uma substring de T que é igual a S . Duas ocorrências de S em T são consideradas diferentes se ocorrem em posições diferentes em T .

Tendo muito o que estudar, Carlos pensou em desistir e contar quantas alucinações diferentes possíveis ele poderia ter, respeitando a restrição de no máximo uma ocorrência de K em B_i . Duas alucinações são diferentes se uma ocorrência aparece em uma alucinação, mas não na outra (lembre-se do que significam ocorrências diferentes no parágrafo anterior).

No entanto, Carlos nunca desiste. Então cabe a você contar isso e satisfazer sua curiosidade! Como esse valor pode ser muito grande, imprima o resto da divisão por 10^9+7 .

Entrada

A primeira linha da entrada contém a palavra-chave K ($1 \leq |K| \leq 2 \cdot 10^5$), que é uma string de letras minúsculas latinas (a-z).

A segunda linha da entrada contém um inteiro N ($1 \leq N \leq 2 \cdot 10^5$): o número de livros.

A i -ésima das N linhas seguintes contém uma string de letras minúsculas latinas B_i ($1 \leq |B_i| \leq 2 \cdot 10^5$), representando o i -ésimo livro.

É garantido que $\sum_{i=1}^N |B_i|$ é no máximo $2 \cdot 10^5$.

Saída

Imprima uma única linha: o resto do número de alucinações possíveis que Carlos pode ter (considerando as restrições do problema) ao dividir por 10^9+7 .

Exemplos

Entrada

```
dp
3
naoseidp
dpsdedpsdeamanha
muitofacildpseitudodedpdpsvoufazerquestao
```

Saída

24

Entrada

```
gaga
3
raraahahahromaromamagagaohlahlah
gagagag
togag
```

Saída

6

Explicação

No primeiro livro, a palavra-chave ``gaga’’ ocorre uma vez, então a alucinação de Carlos pode destacar essa ocorrência ou não destacar nenhuma (2 possibilidades).

No segundo livro, ``gaga’’ ocorre duas vezes, começando em posições diferentes. Portanto, há 3 possibilidades: destacar a primeira ocorrência, destacar a segunda ocorrência ou não destacar nenhuma.

No terceiro livro, ``gaga’’ não ocorre, então há apenas 1 possibilidade.

Considerando todas as possibilidades de destaque para cada livro, obtemos o número total de alucinações possíveis: $2 \cdot 3 \cdot 1 = 6$.

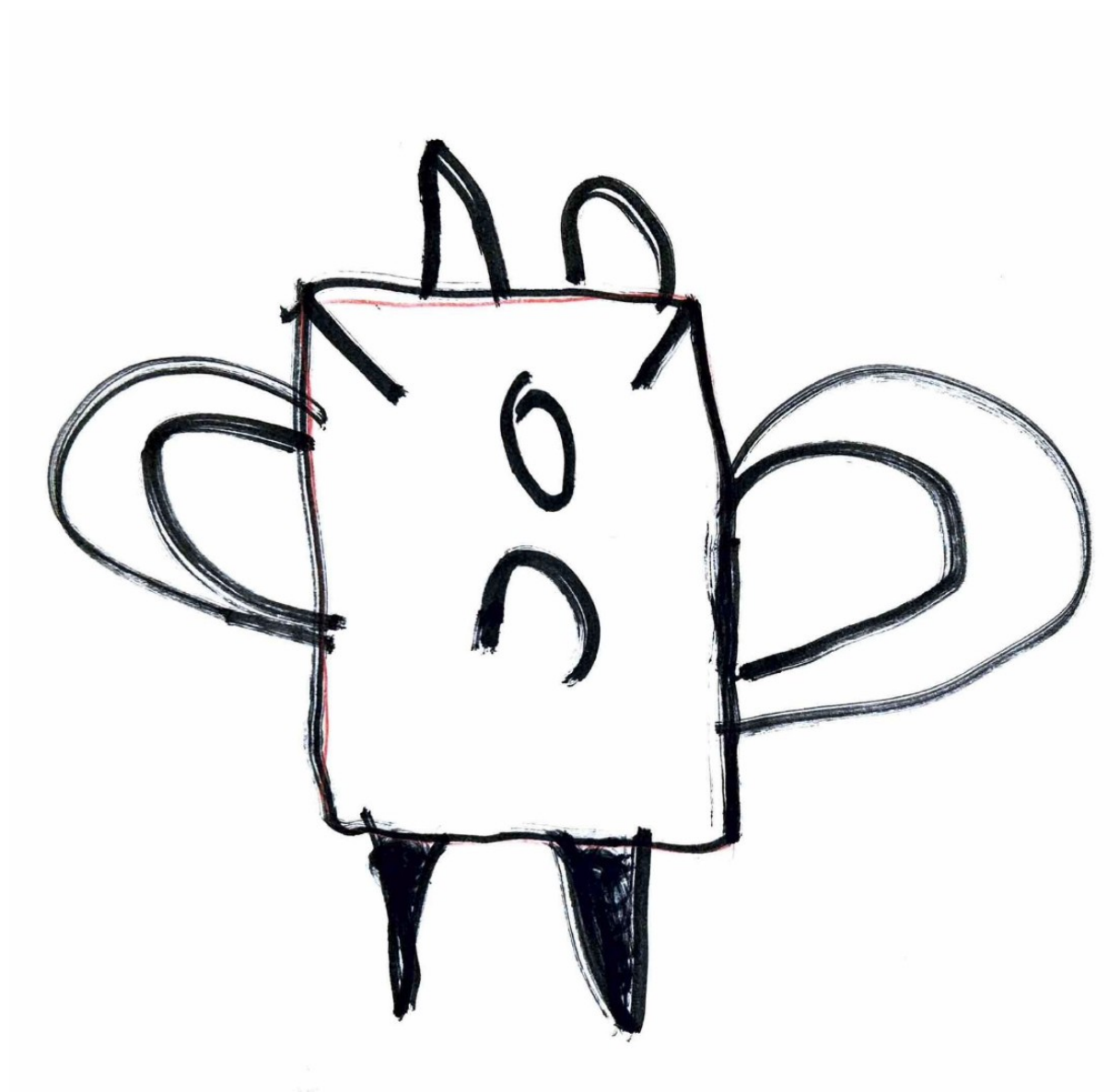
Entrada

```
aaaaaaaaa
5
aaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
a
```

Saída

95760

Problem F — Fada Quadrada



Fada Quadrada

A Fada Quadrada garante a ordem na Intlândia, usando seus poderes matemáticos para impor a quadratura em seu quadrante, afinal, os números quadrados são melhores que os outros.

O propósito da Fada Quadrada é claro: enquadrar os números. Ajude-a a melhorar o mundo dos números mágicos dos números.

Entrada

A entrada consiste de uma quantidade desconhecida de números inteiros $0 \leq i \leq 100$, um por linha, terminando em EOF. É garantido que a entrada de cada caso de teste não contém mais de 100 números.

Saída

A Fada Quadrada transforma cada número lido no número quadrado mais próximo de seu valor, conforme os exemplos.

Exemplos

Entrada

20
7

Saída

16
9

Entrada

2
4
8

Saída

1
4
9

Problem G — Grande Desastre

Após a GatoNet tomar conta do mundo, a humanidade foi forçada a se esconder e lutar contra as máquinas. Existem células de resistência humanas espalhadas pelo mundo todo, tentando, de alguma forma, se organizar e resistir à dominação das máquinas.



As células de resistência se comunicam por meio de mensagens cifradas com um protocolo de comunicação ainda inviolado pelas máquinas. Se uma célula de resistência A se comunica com outra célula B , também é garantido que B consiga se comunicar com A . Além disso, uma célula pode atuar como papel de intermediária entre outras duas células, repassando mensagens de uma para outra. É garantido que uma célula consegue se comunicar com qualquer outra, seja diretamente ou por meio de outras células intermediárias.

As máquinas bolaram um plano para tentar quebrar as comunicações. Elas identificaram as células mais importantes, chamadas de *células críticas*, que são aquelas que, se destruídas, fariam com que a comunicação entre algum par de células fosse interrompida. Sabendo dos planos das máquinas, as células de resistência decidiram se organizar e proteger essas células mais importantes. Para isso, elas resolveram criar um canal alternativo entre pares de células, não necessariamente críticas, de forma que, mesmo que uma célula crítica seja destruída, a comunicação entre as células de resistência não seja interrompida.

Como os recursos são escassos, determine o número mínimo de canais alternativos que devem ser criados para garantir que a comunicação entre todas as células de resistência seja mantida, mesmo que alguma célula crítica seja destruída.

Entrada

A primeira linha da entrada possui dois inteiros n e m ($1 \leq n \leq 2 \cdot 10^5$, $0 \leq m \leq 10^6$), representando o número de células de resistência e o número de canais de comunicação existentes, respectivamente. As células são numeradas de 1 a n . Cada uma das próximas m linhas contém dois inteiros u e v ($1 \leq u, v \leq n$, $u \neq v$), indicando que existe um canal de comunicação direto entre as células u e v .

Saída

Determine o número mínimo de canais alternativos que devem ser criados para garantir que a comunicação entre todas as células de resistência seja mantida, mesmo que alguma célula crítica seja destruída.

Exemplos

Entrada

```
5 6
1 2
2 3
3 1
3 4
4 5
5 3
```

Saída

```
1
```

Entrada

```
14 16
1 2
1 3
2 4
3 4
4 5
5 6
6 7
7 8
8 9
9 10
8 10
9 11
11 12
12 7
12 13
7 14
```

Saída

```
2
```

Problem H — Hugo e o Jogo das Palavras

Hugo tem apenas 6 anos e está iniciando seu processo de alfabetização. Para ensinar o alfabeto para ele e para as demais crianças, a professora propôs a seguinte brincadeira: Hugo escolheria uma palavra s e a professora uma palavra t . Se s anteceder t na ordem alfabética, Hugo e as crianças são os vencedores; caso contrário, a professora vence o jogo.

Lembre-se da definição de ordem alfabética para palavras: para duas palavras diferentes a e b , caso uma seja prefixo da outra, a menor em tamanho antecede a outra nessa ordem. Caso contrário, toma-se a primeira posição (da esquerda para a direita) em que elas diferem: a que tiver o caractere menor (na ordem do alfabeto) nesta posição é a menor palavra na ordem alfabética.

Seguindo seu objetivo didático, a professora também estabeleceu a seguinte regra: antes de comparar as palavras, Hugo pode selecionar uma dentre as duas. Uma vez selecionada a palavra, ele pode escolher duas letras, em posições distintas, e trocá-las de lugar. Hugo pode optar por não fazer nenhuma troca antes da comparação das palavras, mas ele não pode realizar mais de uma troca.

Por exemplo, suponha que Hugo escolheu $s = \text{“maca”}$ e a professora, $t = \text{“doutor”}$. Neste momento, a professora seria a vencedora, mas Hugo selecionou a palavra s e trocou a primeira e a terceira letras de posição, obtendo agora $s = \text{“cama”}$ e vencendo a partida.

A criança gostou da brincadeira. Para ajudar a professora a verificar se as crianças têm ou não chances de vencer a partida, escreva um programa que receba as palavras s e t e decida se existe alguma forma de Hugo e as crianças vencerem a partida.

Entrada

A primeira linha da entrada contém o inteiro n ($1 \leq n \leq 100$), que indica o número de letras na palavra s .

A segunda linha da entrada contém a palavra s , que é composta de n caracteres alfabéticos minúsculos.

A terceira linha da entrada contém o inteiro m ($1 \leq m \leq 100$), que indica o número de letras na palavra t .

A quarta linha da entrada contém a palavra t , que é composta de m caracteres alfabéticos minúsculos.

Saída

Imprima, em uma linha, o caractere ‘S’, caso exista alguma forma de Hugo e as crianças vencerem a partida; caso contrário, imprima o caractere ‘N’.

Exemplos

Entrada

4

```
maca
6
doutor
```

Saída

```
S
```

Explicação

Este foi o exemplo apresentado no texto.

Entrada

```
4
moto
4
caso
```

Saída

```
S
```

Explicação

Hugo pode trocar a primeira e a terceira letra para obter $t = \text{"saco"}$, que sucede "moto" na ordem alfabética.

Entrada

```
4
fcte
3
abc
```

Saída

```
N
```

Entrada

```
1
a
1
a
```

Saída

```
N
```

Explicação

Hugo não consegue fazer trocas neste caso. Como as palavras são iguais, s não antecede t na ordem alfabética, portanto, a professora ganha.

Problem I — Igualzim, não!

Geraldo e Genésio são dois compadres que moram em um pequeno município de Minas Gerais. Ambos resolveram pintar as fachadas de suas residências e, conversando sobre o assunto, descobriram que ambos iriam usar a cor azul. Na mesma hora, Genésio bradou: “Igualzim, não!”.

O problema é que os moradores são muito sistemáticos e também muito próximos, de modo que muitos são compadres uns dos outros. Além disso, na venda do seu Tião, único estabelecimento comercial da região, só há duas cores de tinta disponíveis: azul e branco (todos lá são Cruzeiro...).

Sabendo que dois compadres não pintarão suas casas de uma mesma cor, e dadas as informações sobre os N moradores do município (quem é ou não compadre do outro), determine o número de maneiras distintas de renovar as fachadas das residências da região, uma vez que a notícia se espalhou e ninguém quer ficar de fora (isto é, todos pintarão suas casas).

Entrada

A primeira linha da entrada contém o inteiro N ($1 \leq N \leq 2 \times 10^5$), indicando o número de moradores do município. Cada morador será identificado por um inteiro único no intervalo $[1, N]$.

As N linhas seguintes contêm, cada uma, a lista de compadres. A i -ésima linha começa com um inteiro m_i ($0 \leq m_i < N$), indicando quantos são os compadres do i -ésimo morador. Após um espaço em branco, seguem m_i inteiros c_k ($1 \leq c_k \leq N$, $c_k \neq i$, $k \in [1, m_i]$), separados por um espaço em branco, indicando que c_k é compadre de i .

É garantido que:

- se i é compadre de j , então j também é compadre de i ;
- i ser compadre de j e j ser compadre de k não implica, necessariamente, que i e k sejam compadres;
- $\sum_{i=1}^N m_i \leq 4 \times 10^5$.

Saída

Imprima, em uma linha, o número de maneiras distintas que as casas do município podem ser coloridas, usando as tintas da venda do seu Tião e de modo que compadres tenham casas de cores diferentes. Como este número pode ser muito grande, imprima o seu resto da divisão por $10^9 + 7$.

Exemplos

Entrada

```
3
1 2
```

```
2 1 3
1 2
```

Saída

```
2
```

Explicação

Há dois pares de compadres: 1 e 2, 2 e 3. Existem duas formas de pintar as casas: 1 e 3 pintam suas casas de branco e 2 de azul, ou 1 e 3 pintam suas casas de azul e 2 pinta a sua de branco. Note que 1 e 3 não são compadres, de modo que podem ter casas de mesma cor.

Entrada

```
3
2 2 3
2 1 3
2 1 2
```

Saída

```
0
```

Explicação

Todos os moradores são compadres entre si, de modo que não há como pintar as casas com apenas as duas cores disponíveis sem que ao menos dois compadres fiquem com a mesma cor.

Entrada

```
2
0
0
```

Saída

```
4
```

Explicação

Os dois moradores não são compadres, de modo que cada um pode pintar sua casa da cor que preferir. Assim, são quatro maneiras distintas: AA, AB, BA e BB, se usarmos a letra A para sinalizar a cor azul e B para a cor branca.

Problem J — Memory Game

Every Sunday, Zuleide sets up her stall at the Feira da Torre de TV, between the pastel tent and the hammock one. She sells a handmade memory game, with cards that carry the monuments of Brasília drawn in India ink: two Congresses, two Cathedrals, two JK bridges, and so on. Since nobody buys what they cannot try, Zuleide came up with a challenge to attract customers: whoever clears her board takes the game for free.

The challenge works like this. Zuleide shuffles the cards and spreads them face down on a board with N rows and M columns. Each card carries a drawing, and each drawing appears on exactly two cards of the board. To talk to the customers from out of town, who cannot tell the Itamaraty from the Palácio da Justiça, she numbered the drawings from 1 to $\frac{N \cdot M}{2}$. On each move, the customer points at two cards that are still face down and Zuleide turns them over. If the drawings are equal, the pair has been found and the two cards stay face up until the end of the game. If they are different, Zuleide says which drawing was on each one and turns both back face down.

The catch is the limit: the customer only takes the game by finding every pair using at most as many moves as there are cards on the board, that is, $N \cdot M$ moves. Zuleide guarantees that it is always possible to win. Except that almost nobody manages it, and whoever loses ends up buying the game anyway, which is exactly her plan.

You are in the queue with your money counted out and you have decided that you will take the game for free. Write a program that plays against Zuleide and clears the board within the limit.

This is an interactive problem: instead of reading a fixed input, your program talks to the judge, which plays the role of Zuleide. Your program points at the cards and receives back what she saw when she turned them over.

Interaction

At the start, the judge sends a line with two integers N and M ($1 \leq N, M \leq 10^4$), the dimensions of the board. It is guaranteed that $N \cdot M$ is even and that $2 \leq N \cdot M \leq 10^4$. The rows are numbered from 1 to N , from top to bottom, and the columns from 1 to M , from left to right. Each card carries an integer between 1 and $\frac{N \cdot M}{2}$, and each one of those integers appears on exactly two cards.

On each move, your program prints a line `r1 c1 r2 c2`, with $1 \leq r_1, r_2 \leq N$ and $1 \leq c_1, c_2 \leq M$, indicating the two cards it wants to turn over: the one in row r_1 and column c_1 and the one in row r_2 and column c_2 . The two positions must be distinct and neither of them may be of a card that is already paired. The judge answers with one line:

- `IGUAL` (Portuguese for “equal”), if the two cards carried the same integer. The pair has been found and the two cards stay face up until the end of the game.
- `DIFERENTE v1 v2` (“different”), if they carried different integers: v_1 is the integer of the card at (r_1, c_1) and v_2 is the one of the card at (r_2, c_2) . Both go back to being face down.

When the IGUAL answer completes the last pair, the game is over and your program must finish.

Your program can make at most $N \cdot M$ moves. Going over that limit, pointing at a position outside the board, repeating the same position in the two cards of one move or turning a card that is already paired gives *Wrong Answer*.

It is guaranteed that the board is fixed before the first move, that is, the judge does not move the cards around according to what you played, and that a strategy exists that clears any board within the limit.

Flush the output after each printed line. The conversation with the judge goes through a *pipe*, and a line stuck in the output buffer never reaches the other side: your program is left waiting for an answer that does not come and the verdict ends up being an unfair *Time Limit Exceeded*. Use `fflush(stdout)` in C, `std::endl` or `<< std::flush` in C++, `print(..., flush=True)` in Python, `io::stdout().flush()` in Rust and `System.out.flush()` in Java and Kotlin, whose JVMs have a buffer of their own. In Bash, `echo` already delivers the line. Always read the answer before making the next move, and read one line at a time: reading the input to the end hangs the program, because the judge only speaks after each move of yours.

Input

The only thing your program receives before playing is the line with N and M . Everything it reads afterwards are the IGUAL and DIFERENTE v_1 v_2 answers, one per line, each one right after a move it printed itself.

Output

Your program prints the moves in the format r_1 c_1 r_2 c_2 , one per line, and finishes as soon as the IGUAL answer completes the last pair.

Example

In the dialogues below, the lines without indentation are what your program prints and the indented lines are what the judge answers.

On a 2×2 board whose cards, hidden, are

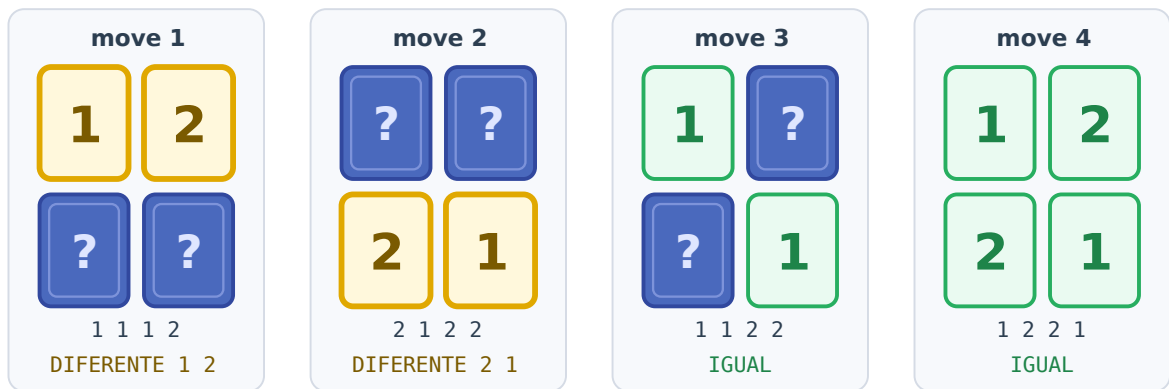
```
1 2
2 1
```

a winning game is:

```
      2 2
1 1 1 2
      DIFERENTE 1 2
2 1 2 2
      DIFERENTE 2 1
1 1 2 2
      IGUAL
1 2 2 1
```

IGUAL

The figure below reflects this game. In the first two moves the cards turned over were different and went back face down; in the last two, the pairs were found. There were 4 moves, exactly the limit $N \cdot M = 4$.



On a 1×2 board there is only one pair, and one move solves it:

1 2
1 1 1 2
IGUAL

Problem K — Kolision

Dani and Pedro are playing with their new convex polygons. After getting tired, they decided to leave them on autopilot, each with a specific velocity.

Let v_x, v_y be the velocity of a polygon P . After a time $t \geq 0$, the whole polygon has moved $v_x \cdot t$ units along the x coordinate and $v_y \cdot t$ units along the y coordinate.

The biggest problem with letting polygons roam is collision, which deforms and wears them out. A collision happens when some point is covered by both polygons.

Dani easily computed when they need to take back control before any accident, but decided to test whether you were paying attention in the last Computational Geometry classes.

Find the instant at which Dani's and Pedro's polygons collide!

Input

The first line contains three integers N, v_{ax}, v_{ay} ($3 \leq N \leq 10^5$, $-1000 \leq v_{ax}, v_{ay} \leq 1000$, $v_{ax}^2 + v_{ay}^2 > 0$) — the number of vertices of A and its velocity.

Each of the next N lines contains two integers $x y$ ($0 \leq x, y \leq 10^9$) — the vertices of A in counter-clockwise order. It is guaranteed that no three points of A are collinear and A is a convex polygon.

The next line contains three integers M, v_{bx}, v_{by} ($3 \leq M \leq 10^5$, $-1000 \leq v_{bx}, v_{by} \leq 1000$, $v_{bx}^2 + v_{by}^2 > 0$) — the number of vertices of B and its velocity.

Each of the next M lines contains two integers $x y$ ($0 \leq x, y \leq 10^9$) — the vertices of B in counter-clockwise order. It is guaranteed that no three points of B are collinear and B is a convex polygon.

It is guaranteed that the polygons do not intersect at time $t=0$.

Output

Print a single real number $t > 0$ — the first instant at which the two polygons collide — or -1 if they never collide.

Your answer is accepted if its absolute or relative error is at most 10^{-6} .

Examples

Input

```
4 1 0
0 0
2 0
2 3
0 3
4 -1 0
10 3
```

```
12 3
13 4
10 4
```

Output

```
4
```

Explanation

In the sample, the rectangles start 8 units apart and approach each other at relative speed 2, first touching at $t = 4$.

Input

```
4 -1 0
0 0
2 0
2 2
0 2
4 1 0
10 0
12 0
12 2
10 2
```

Output

```
-1
```

Explanation

The rectangles are moving apart.

Problem L — Longe de Casa

Na montanha do Peak, n exploradores estão perdidos!

Pedro Gallo e seu fiel companheiro, o Bing Bong, planejam usar um helicóptero para resgatar os exploradores. Ele escolherá uma posição na montanha para pousar, e os exploradores terão que alcançar o helicóptero para serem resgatados.

O mapa da montanha pode ser representado por um plano cartesiano, sendo as posições representadas por pontos de coordenadas inteiras. Cada explorador está em uma posição inicial (x, y) , tem energia k e possui uma unidade de foguete. Com 1 de energia ele pode se mover uma unidade verticalmente ou horizontalmente, ou, ao utilizar seu foguete (ainda utilizando 1 de energia), se mover quanto quiser na direção vertical ou horizontal. Mais formalmente, se a energia do explorador for positiva, ele pode gastar 1 de energia para fazer uma das seguintes ações:

- Ir para $(x+1, y)$, $(x-1, y)$, $(x, y+1)$ ou $(x, y-1)$.
- Se o explorador ainda tiver seu foguete, ir para $(x+d, y)$ ou $(x, y+d)$ para qualquer inteiro d (note que d pode ser negativo). Ao realizar esta ação, o foguete se quebra e não pode mais ser utilizado.

Escolhida a posição do helicóptero, um explorador poderá ser resgatado somente se consegue se mover até ela, a partir de sua posição inicial, com as ações descritas. Ajude Pedro Gallo e Bing Bong e calcule a maior quantidade de exploradores que poderá ser salva ao escolher a posição de pouso de forma ótima.

Entrada

A primeira linha de entrada contém um inteiro n ($1 \leq n \leq 2 \cdot 10^5$) — o número de exploradores.

A i -ésima das próximas n linhas contém três inteiros x_i, y_i, k_i ($1 \leq x_i, y_i, k_i \leq 2 \cdot 10^5$) — a posição inicial (x_i, y_i) do i -ésimo explorador e sua energia k_i .

Saída

Imprima um único inteiro — a maior quantidade de exploradores que pode ser salva escolhendo a posição de pouso do helicóptero de forma ótima.

Exemplos

Entrada

```
3
1 1 1
2 2 1
3 3 1
```

Saída

2

Entrada

3
10 21 2
19 30 2
20 19 1

Saída

3

Entrada

5
1 1 1
1 1 1
100 100 1
100 100 1
100 100 1

Saída

5

Entrada

7
77090 53996 1
198356 90124 4
91506 118145 3
91595 112415 1
1718 180618 3
70396 46316 118358
85973 72452 6

Saída

3

Problem M — Malha de Amplificadores

Uma malha de amplificadores é organizada na forma de uma árvore de N vértices. Cada vértice v contém um amplificador cujo comportamento é determinado por um par de coeficientes inteiros (c_v, d_v) . Quando um sinal de intensidade x passa pelo amplificador do vértice v , sua nova intensidade é

$$c_v x + d_v \pmod{998244353}.$$

Você deve processar Q operações de dois tipos:

- **Consulta:** um sinal de intensidade inicial x percorre o caminho de u até v . Determine sua intensidade final após passar, em ordem, por todos os amplificadores desse caminho, incluindo os dos vértices u e v .
- **Atualização:** substitua os coeficientes de todos os amplificadores no caminho entre u e v , incluindo os extremos, pelos valores c e d fornecidos.

Note que, em uma consulta, o amplificador de u é aplicado primeiro e o de v por último. Quando $u=v$, o sinal passa uma única vez pelo amplificador desse vértice.

Entrada

A primeira linha contém dois inteiros N e Q ($1 \leq N, Q \leq 200000$), o número de vértices e o número de operações, respectivamente.

Cada uma das próximas N linhas contém dois inteiros c_v e d_v ($0 \leq c_v, d_v < 998244353$), descrevendo os coeficientes iniciais do amplificador do vértice v , para $v=1, 2, \dots, N$.

Cada uma das próximas $N-1$ linhas contém dois inteiros u e v ($1 \leq u, v \leq N$), indicando uma aresta não direcionada entre esses vértices. As arestas formam uma árvore.

Cada uma das próximas Q linhas descreve uma operação em um dos seguintes formatos:

- **1 u v x:** determine a intensidade final de um sinal que começa com intensidade x e percorre o caminho de u até v , onde $0 \leq x < 998244353$.
- **2 u v c d:** para cada vértice w no caminho entre u e v , substitua seus coeficientes (c_w, d_w) por (c, d) , onde $0 \leq c, d < 998244353$.

Em todas as operações, $1 \leq u, v \leq N$.

Saída

Para cada operação do tipo 1, imprima uma linha contendo um único inteiro: a intensidade final do sinal módulo 998244353.

Exemplos

Entrada

```
2 5
2 1
3 4
```

```
1 2
1 1 2 2
1 2 1 2
2 1 1 1 5
1 1 2 3
1 1 1 0
```

Saída

```
19
21
28
5
```

Explicação

Na primeira consulta, o sinal começa com intensidade 2. Ao passar pelo vértice 1, multiplicamos sua intensidade por 2 e somamos 1, tornando-se 5. Em seguida, no vértice 2, ela é multiplicada por 3 e acrescida de 4, chegando a 19.

Na segunda consulta, o sinal também começa com intensidade 2, mas percorre o caminho no sentido inverso: sai do vértice 2 com intensidade 10 e, após passar pelo vértice 1, chega à intensidade final 21.

A terceira operação ajusta apenas o amplificador do vértice 1, que passa a somar 5 à intensidade recebida.

Entrada

```
5 7
1 1
2 0
1 3
3 1
1 2
1 2
1 3
3 4
3 5
1 2 4 2
2 2 4 1 1
1 2 4 0
1 5 4 3
2 1 5 2 0
1 2 4 4
1 5 2 1
```

Saída

```
25
4
7
```

21 9

Explicação

A segunda operação ajusta os amplificadores dos vértices 2, 1, 3 e 4 para que cada um some 1 à intensidade recebida. Na consulta seguinte, o sinal começa com intensidade 0 e passa por esses mesmos vértices: sua intensidade passa a ser 1, 2, 3 e, por fim, 4.

Problem N — Nome de Usuário

Nomes de usuário legais no estilo dos anos 2000 usam palavras imaginativas, anos de nascimento e letras extras.

A idéia era mostrar sua rebeldia, ousadia e o quão descolado a gente poderia ser.

Para isso a gente misturava palavras do nosso agrado, com referências à músicas, artistas, nomes de personagens e termos em outros idiomas. Trocadilhos eram bem-vindos, principalmente se eram engraçados.

Para um amante de senhor dos anéis, uma idéia seria usar o nome do personagem Aragorn e brincar com a palavra.

Por exemplo, trocar ‘A’ por ‘4’, ‘O’ por ‘0’ e assim ter 4r4g0rn. Ficou legal? Não, mas aí já é outra história...

Crie novos nomes de usuários legais à partir de valores de entrada. Siga as regras:

1. Troque todas as ocorrências de ‘A’ ou ‘a’ para ‘4’.
2. Troque todas as ocorrências de ‘O’ ou ‘o’ para ‘0’.
3. Troque todas as ocorrências de ‘E’ ou ‘e’ para ‘3’.
4. Troque todas as ocorrências de ‘S’ ou ‘s’ para ‘\$’.

Além da preocupação com o estilo, o tamanho também é importante. Você deve rejeitar nomes de usuário que não tiverem de 4 a 20 caracteres.

Entrada

Em cada caso de teste, a entrada consiste em um único nome de usuário S em formato de texto. S é uma cadeia de caracteres formada apenas por letras minúsculas e/ou maiúsculas de a/A a z/Z, com $1 \leq |S| \leq 30$.

Saída

Se o nome de usuário da entrada não respeitar o limite de tamanho (entre 4 e 20), imprima: “ S com tamanho incorreto: $|S|$ ” (sendo S a cadeia de entrada e $|S|$ o seu tamanho). Caso respeite, imprima, em uma única linha, a sugestão de nome de usuário legal.

Exemplos

Entrada

```
Aragorn
```

Saída

```
4r4g0rn
```

Entrada

```
Gandalf
```

Saída

```
G4nd4lf
```

Entrada

```
Gimli
```

Saída

```
Gimli
```

Entrada

```
abc
```

Saída

```
abc com tamanho incorreto: 3
```